Neuro-Evolved Heuristics for Variable Gapped Common Subsequence Identification

Marko Djukanović[1,5], Christian Blum[2], Aleksandar Kartelj[3], Sašo Džeroski[4], and Žiga Zebec[5]

[1] University of Nova Gorica, Nova Gorica, Slovenia
marko.dukanovic@ung.si
[2] Artificial Intelligence Research Institute (IIIA-CSIC), Barcelona, Spain
christian.blum@iia.csic.es
[3] Faculty of Mathematics, University of Belgrade, Belgrade, Serbia
kartelj@matf.rs
[4] Jožef Stefan Institute, Ljubljana, Slovenia
saso.dzeroski@ijs.si
[5] Institute of Information Sciences (IZUM), Maribor, Slovenia
ziga.zebec@izum.si

Abstract. This study addresses the Variable Gapped Longest Common Subsequence (VGLCS) problem, a variant of the classical longest common subsequence problem that incorporates additional gap constraints. The problem has applications in sequence alignment and time-series analysis. While the two-sequence version has been extensively studied using dynamic programming, its generalized form with multiple input sequences is typically tackled using beam search–based heuristics. However, existing hand-crafted heuristics often suffer from limited robustness.

To overcome this issue, we propose a learning-based approach for automatically designing more effective data-driven heuristics. The heuristics are modeled by a neural network with a predefined architecture, whose weights are optimized by a genetic algorithm in a neuro-evolutionary framework. The learning process alternates between evolutionary optimization of network weights and their evaluation within an iterative multi-source beam search procedure, a state-of-the-art method for the tackled problem. Importantly, the neural network does not directly construct solutions but instead learns to guide the search process, resulting in a neuro-evolved heuristic. Furthermore, we introduce an ensemble heuristic that combines the learned heuristics with the best-performing hand-crafted heuristic. The resulting hybrid approach, integrated into the iterative multi-source beam search framework, outperforms existing methods on both synthetic benchmark instances from the literature and the newly introduced real-world instances with data-driven gap constraints.

Keywords: Beam search; Neural networks; Neuro-Evolved heuristics; Gap constraints

1 Introduction

The *Longest Common Subsequence Problem* (LCSP) [5] is a fundamental combinatorial optimization problem with numerous applications in computational biology, particularly in the analysis of molecular sequences. Given a set of input sequences $S = \{s_1, \ldots, s_m\}$ over an alphabet Σ, the objective is to determine the longest subsequence common to all sequences $s_i \in S$. Over the past decades, several practically motivated variants of LCSP have been proposed by incorporating additional structural or biological constraints, including the constrained, arc-preserving, and repetition-free variants [8, 10].

In this work, we study the *Variable Gapped Longest Common Subsequence Problem* (VGLCSP), introduced in [13], theoretically analyzed in [1], and recently evaluated experimentally in [4]. This variant extends the classical LCSP by introducing flexible gap constraints between consecutive symbols in the subsequence, allowing these gaps to vary along the input sequences.

Formally, for each sequence $s_i \in S$, a gap function $G_{s_i} : \{1, \ldots, |s_i|\} \to \mathbb{N}$ specifies the maximum allowed distance between consecutive matched positions. If a solution s appears at positions $i_i^1 < \cdots < i_i^{|s|}$ in s_i, the gap constraints are satisfied if

$$i_i^x - i_i^{x-1} \leq G_{s_i}[i_i^x] + 1, \quad \forall x = 2, \ldots, |s|.$$

A subsequence is feasible if these constraints hold for all sequences $s_i \in S$. The objective is to find the longest feasible subsequence. The VGLCSP provides a flexible and realistic model for sequence alignment, particularly relevant for DNA and protein analysis with variable structural distances, as well as for time-series analysis where events must occur within specified temporal delays [9].

While several exact dynamic programming approaches exist for the two-sequence case ($m = 2$) [12], the problem becomes computationally intractable for larger m; in fact, it is NP-hard in the general case. To date, no efficient exact methods are known for arbitrary m. The only available scalable approach is an iterative multi-source beam search (IMSBS) [4], which explores a state-space representation of the problem. The method iteratively refines candidate root nodes and extends them through feasible symbol additions. Despite its scalability, IMSBS relies on hand-crafted heuristics, which tend to degrade in performance as m increases or the alphabet size $|\Sigma|$ decreases [5]. The presence of gap constraints further exacerbates this issue.

In addition, existing studies are limited to synthetic benchmark instances, leaving a gap in evaluating methods on realistic data.

To address these limitations, we propose a learning-enhanced beam search framework for VGLCSP. Our approach integrates learned and hand-crafted heuristics into a unified search strategy. The learned heuristic is represented by a multi-layer neural network that scores candidate states during the search. The network parameters are optimized through a genetic algorithm in a neuro-evolutionary framework, where each candidate model is evaluated within the IMSBS procedure. Importantly, the neural model does not directly construct solutions but learns to guide the search process.

Furthermore, we design an ensemble heuristic that combines the learned heuristic with the best-performing hand-crafted heuristic, yielding a more robust guidance mechanism. This hybrid approach is inspired by recent advances in learning-guided combinatorial optimization [6, 15, 19], but differs in explicitly leveraging complementary heuristic signals within an ensemble.

Finally, to address the lack of realistic benchmarks, we introduce new problem instances derived from biologically motivated datasets, augmented with data-driven gap constraints.

Contributions. The main contributions of this work are:

- A neuro-evolutionary framework integrated for learning heuristics in VGLCSP.
- An ensemble heuristic combining learned and hand-crafted guidance.
- An enhanced IMSBS algorithm integrating the proposed heuristics.
- A new set of realistic benchmark instances with data-driven gap constraints.

The paper is organized as follows. In Section 2, the iterative multi source beam search approach from the literature is roughly presented. Section 3 provides main details on the process of learning heuristic guidance, and the design of ensemble heuristic estimator for the tacked problem. Section 4 reports a rigorous experimental analysis. Finally, Section 5 concludes the paper highlighting several future research directions.

2 Iterative Multi-Source Beam Search for the VGLCSP

The first scalable approach for the VGLCSP with an arbitrary number of sequences (m) was introduced in [4] as an *Iterative Multi-Source Beam Search* (IMSBS). In this section, we summarize its main components; interested readers are referred to the original paper for full details.

IMSBS is based on a *state graph* representation of the VGLCSP. Each node represents a partial feasible subsequence, characterized by a vector of positions defining a subproblem and the length of the constructed subsequence. Formally, a state $v = (\mathbf{p}^{L,v}, l^v)$ is induced by a partial solution s^v if: (i) $p_i^{L,v} - 1$ is the smallest index such that s^v is a subsequence of the prefix $s_i[1] \cdots s_i[p_i^{L,v} - 1]$; (ii) $l^v = |s^v|$; and (iii) all gap constraints G_{s_i} are satisfied by s^v.

A directed arc $\alpha = (v_1, v_2)$ labeled by $lett(\alpha) = a \in \Sigma$ exists if $l^{v_2} = l^{v_1} + 1$ and $s^{v_2} = s^{v_1} \cdot a$. To expand a state v, all letters occurring in the suffixes $S[\mathbf{p}^{L,v}]$ are considered. For each letter a, the smallest feasible positions $p_i^{L,a} \geq p_i^{L,v}$ are identified such that the gap constraints are satisfied, i.e., $p_i^{L,a} - p_i^{L,v} \leq G_{s_i}(p_i^{L,a})$. The resulting child state (if not dominated) is $w = (\mathbf{p}^{L,a} + 1, l^v + 1)$. Terminal states correspond to non-extendable feasible subsequences.

Unlike the classical LCSP, the VGLCSP admits multiple (potentially exponentially many) root nodes, since starting positions are unconstrained. This may lead to disconnected regions in the state graph. For instance, consider $s_1 = \texttt{ATGGAAAA}$ and $s_2 = \texttt{ATCCAAAA}$ with $G_{s_1} = G_{s_2} = 1$. The root node $((1, 1), 0)$

cannot reach states with position vector $(5, 5)$, thus missing the optimal subsequence `AAAA`. This phenomenon is illustrated in Fig. 1, where the state graph decomposes into disconnected components.

This observation motivates the IMSBS framework, which operates over a dynamically generated set of root nodes explored iteratively. At each iteration, beam search (BS) is applied to selected root nodes. BS is a breadth-first heuristic search strategy [18] that expands a limited number of nodes per level, controlled by the beam width β, and guided by a heuristic function h.

In IMSBS, node selection relies on an upper-bound heuristic defined as

$$\texttt{UB}(v) = l^v + \sum_{a \in \Sigma} \min_{i=1,\ldots,m} |s_i[p_i^{L,v}, |s_i|]|_a,$$

which estimates the maximum achievable extension from state v. For each letter $a \in \Sigma$, the heuristic evaluates its remaining occurrences in each suffix and aggregates these values to derive an upper bound on the achievable subsequence length.

Despite its scalability, IMSBS relies on hand-crafted heuristics whose effectiveness degrades in challenging settings (e.g., large m or small alphabets), especially under gap constraints. On top of IMSBS, we develop a learning-based methodology that enhances the search process of the IMSBS through learned heuristic guidance and its integration with existing IMSBS.

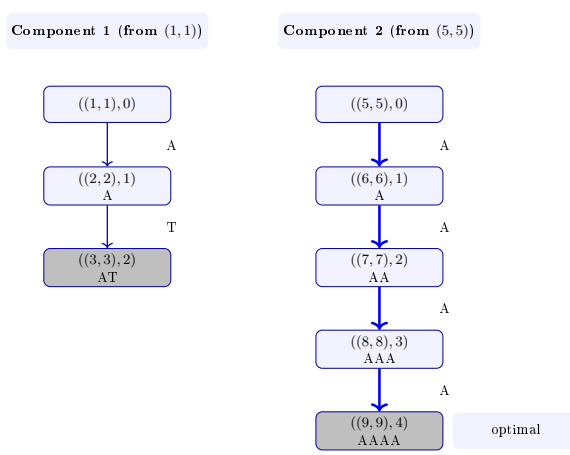

Fig. 1. State graph for $s_1 =$ `ATGGAAAA` and $s_2 =$ `ATCCAAAA` with $G_{s_1} = G_{s_2} = 1$. The state graph consists of two disconnected components. Starting from $(1, 1)$ leads to a suboptimal solution, while starting from $(5, 5)$ yields the optimal subsequence `AAAA`.

In the subsequent part of the paper, we refer to the algorithm as `IMSBS`, returning an approximate solution s_{imsbs}.

3 Learning to Guide the Search

Neural networks (NNs) are well-established machine learning models [7, 3] that have demonstrated strong performance across a wide range of domains. They range from simple multi-layer perceptrons (MLPs) to more complex convolutional and recurrent architectures [11]. In this work, we focus on fully connected multi-layer perceptrons to design an alternative, data-driven heuristic for guiding the IMSBS procedure [4].

The key challenge lies in parameterizing the neural model such that it provides meaningful guidance during the search. Specifically, the network is trained to assign a score to each node (i.e., subproblem), based on features extracted from the corresponding partial solution and its associated residual problem.

In standard neural network training, parameters are optimized via forward and backward propagation, typically using gradient-based methods such as (stochastic) gradient descent [2]. However, in our setting—characterized by a combinatorial NP-hard problem—target values are not readily available, especially for large instances. This limits the applicability of supervised learning approaches. Instead, we adopt a population-based metaheuristic approach, where candidate NN configurations are evaluated indirectly through their impact on the performance of the IMSBS algorithm.

Predefined NN Architecture. The heuristic is modeled as a multi-layer perceptron consisting of an input layer, several hidden layers with user-defined sizes, and a single output neuron. The input layer receives a vector of numerical features describing the current search state, while the output represents the estimated promise of expanding that state.

For each layer l, forward propagation is defined as

$$\mathbf{z}^{(l+1)} = W^{(l)}\mathbf{a}^{(l)} + b^{(l)}, \quad \mathbf{a}^{(l+1)} = \phi(\mathbf{z}^{(l+1)}),$$

where $W^{(l)}$ and $b^{(l)}$ denote the weights and biases, and ϕ is a non-linear activation function (e.g., `tanh`, `ReLU`, or `sigmoid`). The final pre-activation value is used as the heuristic score. All parameters are flattened into a single vector $w = \texttt{flatten}(W, b)$, which represents an individual in the evolutionary process.

Fitness Evaluation. Given a candidate weight vector w, the evaluation proceeds as follows: (i) the weights are loaded into the NN architecture $\mathcal{N}$; (ii) IMSBS is executed on each instance from a training set $\mathcal{T}$ using the neural heuristic h_w; and (iii) the best solution quality obtained per instance is recorded. The fitness of w is defined as the average solution quality over $\mathcal{T}$:

$$\max_{w} \; \mathit{Fitness}(w) = \frac{1}{|\mathcal{T}|} \sum_{I \in \mathcal{T}} f(\texttt{IMSBS}(I, h_w)), \tag{1}$$

where $f(\cdot)$ denotes the length of the best subsequence found. Validation performance is computed analogously on a separate dataset $\mathcal{V}$.

Feature Extraction. To guide the search effectively, each node $v = (\mathbf{p}^v, l^v)$ is mapped to a fixed-length feature vector capturing both the construction progress

and the structure of the induced subproblem. The design is instance-independent and robust across varying problem sizes.

Let $S = \{s_1, \ldots, s_m\}$ denote the input sequences. The position vector $\mathbf{p}^v$ is first normalized by sequence lengths:

$$F_i^v = \frac{p_i^v}{|s_i|}, \quad i = 1, \ldots, m,$$

ensuring scale invariance. From this normalized vector, we extract four statistics: $\max(\mathbf{F}^v)$, $\min(\mathbf{F}^v)$, $\texttt{mean}(\mathbf{F}^v)$, and $\texttt{std}(\mathbf{F}^v)$, capturing the progress and imbalance across sequences. The current subsequence length l^v is also included.

To reflect future feasibility, we compute gap-related statistics:

$$G^{\min},\ G^{\max},\ G^{\texttt{avg}},\ G^{\texttt{std}},$$

evaluated over the remaining positions in all sequences. These features quantify the flexibility of future extensions: larger gaps indicate more freedom, while smaller or highly variable gaps suggest tighter constraints.

Finally, two global features—alphabet size $|\Sigma|$ and the number of sequences m—are included. All features are standardized to zero mean and unit variance prior to training. The resulting 11-dimensional representation provides a compact yet expressive description of each search state.

Optimization of NN Parameters. To optimize the NN parameters, we employ a *Biased Random-Key Genetic Algorithm* (BRKGA), a population-based evolutionary method. Each individual encodes a candidate weight vector w, initialized uniformly at random in the interval $[-q, q]$ with $q = 1$. The evolutionary process iteratively refines the population toward high-quality heuristics $h_w = \mathcal{N}(w)$, where $\mathcal{N}$ is a predefined NN architecture, based on the fitness defined in Eq. (1). The overall learning mechanism is summarized in Algorithm 1.

Overall Learning Procedure. The proposed learning framework alternates between evolutionary optimization of neural network parameters and their evaluation within the IMSBS procedure. Importantly, the neural network does not directly construct solutions; instead, it learns to *guide* the search process. This places the approach within the class of neuro-evolutionary hyper-heuristics.

Given a predefined NN architecture $\mathcal{N}$, IMSBS parameters, and BRKGA parameters, the procedure begins by generating an initial population P of candidate weight vectors. Each individual is evaluated on the training set $\mathcal{T}$ using IMSBS guided by the corresponding neural heuristic. The best-performing solution is stored as w^*. In our implementation, IMSBS is executed with parameters $\beta = 500$ and $\texttt{beam_iters} = 100$, while the remaining parameters follow [4]. This configuration provides a suitable balance between computational cost and robustness during training.

The evolutionary process proceeds iteratively as follows. The population is first sorted according to fitness. A new population P' is then constructed by preserving the top n_{elite} individuals (elitism), introducing n_{mutants} randomly generated individuals (diversification), and generating $n_{\text{offspring}}$ offspring via biased crossover. In each crossover, one parent is selected from the elite set and the

Algorithm 1 GA-Based Learning of a Beam Search Heuristic

Input: NN architecture $\mathcal{N}$, IMSBS parameters, BRKGA parameters, training set $\mathcal{T}$, validation set $\mathcal{V}$
Output: optimized weight vector w^*
Initialize population P with random weight vectors
Evaluate all individuals on $\mathcal{T}$
$w^* \leftarrow$ best individual in P
while time limit $t_{\max}$ not exceeded **do**
  Sort P in descending order of *Fitness*
  Initialize new population P'
  Copy top n_{elite} individuals from P to P'
  for $i = 1$ to n_{mutants} **do**
    Generate random individual x
    Evaluate x on $\mathcal{T}$ and add to P'
    Update w^* if improved and validation does not degrade
  end for
  for $i = 1$ to $n_{\text{offspring}}$ **do**
    Select one elite parent and one non-elite parent
    Generate offspring via biased crossover (p_{bias})
    Evaluate offspring on $\mathcal{T}$ and add to P'
    Update w^* if improved and validation does not degrade
  end for
  $P \leftarrow P'$
end while
return w^*

other from the remaining population. Each gene (weight) is inherited from the elite parent with probability $p_{\text{bias}} \geq 0.5$, and from the second parent otherwise.

Each newly generated individual is evaluated on $\mathcal{T}$. Whenever an improved solution is found, its performance is additionally assessed on the validation set $\mathcal{V}$. To mitigate overfitting, we adopt an early stopping strategy: if the validation performance deteriorates, the update is discarded.

The process repeats until a time limit $t_{\max}$ is reached (set to 2 CPU hours). The algorithm returns the best weight vector w^*, corresponding to the neural network $\mathcal{N}(w^*)$ that induces the most effective heuristic h_w.

In our experiments, the BRKGA parameters are set to $n_{\text{pop}} = 20$, $n_{\text{elite}} = 1$, $p_{\text{bias}} = 0.5$, and $n_{\text{mutants}} = 7$. A visualization of the learning process is provided in Fig. 2.

3.1 An ensemble approach

According to the preliminary results, the learning heuristics performs in overall better than the competitor hand-crafted heuristic from the literature. However, not for all cases, and especially on the instances with small m. In order to improve the results further, we employ a rank-based ensemble strategy that combines a learned heuristic with an analytical hand-crafted upper bound (UB) [4], see

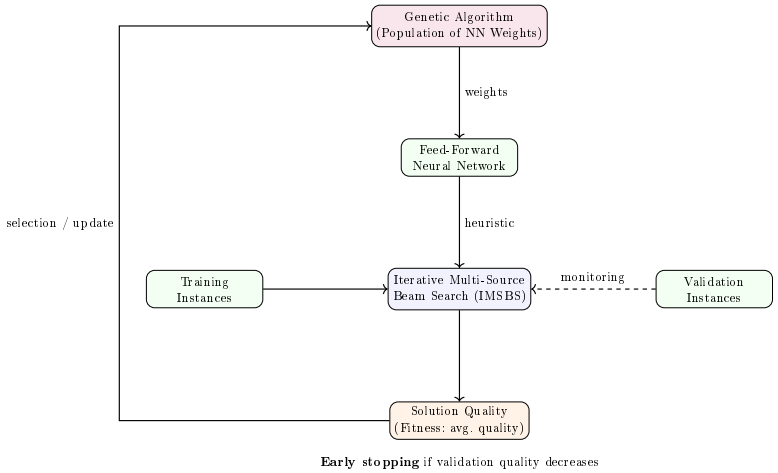

Fig. 2. Enhanced learning framework integrating a Genetic Algorithm with IMSBS. The GA evolves neural network weights, which define a heuristic used within the beam search. Training instances guide optimization, while validation instances monitor generalization. The feedback loop updates the population based on solution quality.

Section 2. Specifically, each candidate node in beam search is evaluated using two criteria: a neural score obtained from a FFNN $\mathcal{N}(w^*)$, denoted $h_{w^*} = \mathcal{N}(w^*)$, and a problem-specific hand-crafted UB.

Instead of combining the raw scores directly, which may differ in scale and distribution, we adopt a rank aggregation approach. First, candidate nodes are sorted independently according to each criterion, producing two rankings. Each node is then assigned a rank based on its position in each sorted list. The final ranking score is computed as a weighted combination of the two ranks:

$$r(v) = \alpha \cdot r_{h_{w^*}}(v) + (1 - \alpha) \cdot r_{\mathrm{UB}}(v), \tag{2}$$

Nodes are then sorted in ascending order of $r(v)$, and the top candidates are retained according to the beam width. This rank-based ensemble heuristic provides a robust mechanism for integrating learned and analytical guidance, mitigating scale inconsistencies and improving search stability.

4 Experimental Evaluation

In this section we compare the performance of the two methods: (i) Imsbs: a baseline state-of-the-art iterative beam search strategy from [4], and (ii) Limsbs-ensamble: a Imsbs guided by (2). All approaches are implemented in C++ and compiled via GCC compiler, evaluated on the VEGA HPC system, hosted at IZUM, Maribor, Slovenia. The cluster consists of 960 compute nodes equipped with the AMD EPYC 7H12 CPUs operating at 2.35 GHz. All experiments were conducted in a single-threaded setting.

Benchmark instances. For the evaluation purposes, two type of benchmark sets are used. The first set of synthetic instances is proposed in [4]. For each combination of $n \in \{50, 100, 200, 500\}$, $m \in \{2, 3, 5, 10\}$, and $|\Sigma| \in \{2, 4\}$, 10 random problem instances are randomly generated, a total of *320 problem instances*. The benchmark set is denoted as RANDOM. Apart from these, a set of biologically-motivated VGLCSP instances are introduced here.

In the context of real instances, we will be modeling context-dependent importance of positions of sequences that represents gap values. It aligns with sequence alignment scoring and weighted subsequence problems. Interpretation of these context relies on the fact that higher weights on positions refer to stronger influence of these nucleotides in forming reliable connections with other (neighboring) acids in common subsequences. Stability ranking between pairs of nucleotides is (statistically) determined in the fundamental works on electrostatic interactions and structural influence [16, 17], used here in creating energy-aware LCS solutions, i.e., structure-aware variable-gap sequences (data-driven constraints):

`DINUC_WEIGHT` $= \{$'AA$'$: 1.00, 'TT$'$: 1.00, 'AT$'$: 0.88, 'TA$'$: 0.58, 'CA$'$: 1.45, 'TG$'$: 1.45, 'GT$'$: 1.44, 'AC$'$: 1.44, 'CT$'$: 1.28, 'AG$'$: 1.28, 'GA$'$: 1.30, 'TC$'$: 1.30, 'CG$'$: 2.17, 'GC$'$: 2.24, 'GG$'$: 1.84, 'CC$'$: 1.84$\}$ 'AA' and its score 1.00 means the stability of interactions between nucleotides `A` and `A` assign a value of 1.0. In essence, value of gap constraint at each position will be proportional to local thermodynamic stability estimated via nearest-neighbor interactions around the nucleotide placed at that position. *Dinucleotide-based structural model* is defined as a dinucleotide weight function: $f(s[i], s[i+1]) = -\Delta G(s[i], s[i+1])$, where ΔG denotes the nearest-neighbor free energy estimated by `DINUC_WEIGHT`.

To ensure consistency with the directional nature of the gapped LCS constraint, a *causal* positional contribution is derived. Each position k of a sequence s is associated only with the interaction formed with its immediate predecessor:

$$f_k^s = \begin{cases} f(s[k-1], s[k]), & k > 1, \\ 0, & k = 1. \end{cases}$$

Each position inherits structural influence from the bond linking it to the previous nucleotide. First position has no predecessor and is therefore assigned zero contribution. This definition enforces a left-to-right dependency structure, ensuring that all subsequent computations of influence and gap constraints depend only on previously observed positions. This is consistent with the feasibility condition in the gapped LCS problem, where admissibility of extending a subsequence depends on the last selected position.

The influence at position i is computed using only preceding positions: $I^s(i) = \sum_{k=\max(1,i-r)}^{i} e^{-\lambda(i-k)} \cdot f_k^s$. This formulation reflects the fact that, in the gapped LCS problem, feasibility of selecting position i depends on the previously selected position. Note that r is a parameter. The resulting influence values induce *directional structure* in the sequence: (i) regions preceded by strong interactions accumulate high influence, (ii) regions following weak or irregular segments exhibit lower influence.

In order to normalize these values, we employ the following formula:

$$\hat{I}_i^{(j)} = \frac{I_i^{(j)} - \min(I^{(j)})}{\max(I^{(j)}) - \min(I^{(j)}) + \epsilon} \in [0,1]\,,$$

and gap constraints are obtained via a *direct mapping*: $w_i^{(j)} = w_{\min} + (w_{\max} - w_{\min}) \cdot \hat{I}_i^{(j)}$. In this context, high influence refers to large allowed gap, whilst low influence refers to restrictive gap.

This mapping reflects the idea that positions with strong structural connectivity can support longer-range dependencies allowing more flexible matching. Interval $[w_{\min}, w_{\max}]$ is parametrized, here fixed to 1 and 10, respectively.

The described instance generator is based on the following parameters: r as influence radius, λ as decay factor, $[w_{\min}, w_{\max}]$ as gap bounds. We additionally fixed $r = 10$, and $\lambda = 0.5$. The resulting benchmark enable systematic evaluation of multi-sequence gapped LCS algorithms under diverse and challenging conditions.

To demonstrate effectiveness of the designed approaches on real-world instances, we have used well-known LCS benchmark sets Rat and Virus [5]. For each input sequence in each problem instance, we have induced a respective data-driven gap constraint by the above methodology. We consider only the instances with $|\Sigma| = 4$ in both datasets, as the gap involvement for the instance with $|\Sigma| = 20$ induced a rather short solutions found by the approaches—therefore hardly relevant from the biological standpoint. The generated dataset, denoted by Real, comprises $2 \cdot 10 = 20$ problem instances. All sequences have a length of 600, and, as already mentioned, differentiate by m, ranging from 10 to 200.

All benchmark sets are freely available at XX. TODO: Revise

Tuning process. Imsbs parameters are tuned on benchmark set Random by employing `irace` [14], with an allowed training budget of 5000 runs (each run is limited to an execution time of 30 minutes) producing the following Imsbs configuration: $\beta = 3000$, $h = \text{UB}$, and $\#imbs_iters = 5000$. One instance from each group has been used for the training, thus 32 problem instances in overall.

All other (less sensible) parameters are set to the values reported in [4]. Concerning the learning Limsbs-ensemble approach, it inherits the same setting form Imsbs, to be completely fair with the comparison. For the learning process, we experimented with different architectures for NN: 2-layer structures with 5 and 10 neurons per layer, and 3-layer configurations with 5 and 10 neurons.

In order to reduce time for tuning different neural network architecture, we allow setting up the same activation function at each layer, selecting among three (most) known: $\{\texttt{tanh}, \texttt{ReLU}, \texttt{sigmoid}\}$. To measure the sensibility analysis w.r.t. set of (11) involved features, we have split them into three groups: (i) Feature configuration 1 F_1 involves 4 features extracted from input sequences, 4 features extracted from gap constraints, and length of partial solution; (ii) When adding $|\Sigma|$ to F_1, we get feature configuration F_2; (iii) When adding m to F_2, we obtain the feature configuration F_3.

After exhaustive training evaluations, we opt to chose the following configurations for the architecture of neural networks, one per different m in the case of benchmark set RANDOM:

- $m = 2$: units at each layer of FFNN is (10, 5, 5); activation function is `ReLu`; and the feature set F_2;
- $m = 3$: units at each layer of FFNN set to (10, 10, 5); activation function is set to `ReLu`; and the feature set to F_2;
- $m = 5$: units at each layer of FFNN set to (10, 5, 5), activation function set to `sigmoid`, and the feature set to F_1;
- $m = 10$: units at each layer of FFNN set to (10, 5, 5); activation function set to `ReLu`, and the feature set to F_2.

The parameter $\alpha = 0.45$ in our ensemble approach except for $m = 3$ where $\alpha = 0.5$ as the results of grid search. We employed 32 instances for the training instances $\mathcal{T}$ and the same amount for the validation instances $\mathcal{V}$; one instance is selected randomly per each (32) group from benchmark set RANDOM. Thus, eight instances are used per each learning process—split by different m—for training and validation purposes.

For the benchmark set REAL, the employed architecture of NN is the same utilized for the benchmark set RANDOM and the largest $m = 10$ case. This is because we wanted to test the robustness of the learned heuristic on other than random instance distributions with a large number of sequences (m). Note that all 20 instances have 10 or more sequences. The parameters of IMSBS follows those settings from the benchmark set RANDOM.

4.1 Numerical results on benchmark set RANDOM

These results are presented in Table 1 which consists of two blocks: the first gives an instance group description, the next block is reserved for the results of LIMSBS-ensemble and IMSBS approaches, reporting the average results of each group (over 10 instances).

Additionally, we evaluated the difference in solution quality between LIMSBS-ensemble and IMSBS using the Wilcoxon signed-rank test on paired instances, reported in Table 2.

The test reveals a statistically significant difference in favor of LIMSBS-ensemble, indicating that the learned approach achieves superior performance overall. Out of 32 instances, LIMSBS-ensemble outperforms IMSBS in 20 cases, while 7 instances result in ties and only 5 favor IMSBS.

4.2 Numerical results on benchmark set REAL

The results are reported in Table 3, organized in a similar manner as Table 1 (with additional columns, reporting the runtimes of each approach per instance ($t[s]$) and lengths of the best LCS from the literature [5] (LCS*)). The following conclusions are drawn.

Table 1. Comparison of objective values between IMSBS and LIMSBS-ensemble on benchmark set RANDOM. Best results per instance category are displayed in bold.

m	n	$\|\Sigma\|$	LIMSBS-ensemble	IMSBS
2	50	2	37.3	**37.7**
2	50	4	**30.1**	**30.1**
2	100	2	72.4	**72.9**
2	100	4	**62.0**	61.9
2	200	2	**142.5**	139.3
2	200	4	**125.9**	125.5
2	500	2	**305.9**	282.1
2	500	4	**305.5**	303.0
3	50	2	**31.2**	**31.2**
3	50	4	**22.9**	**22.9**
3	100	2	**58.5**	**58.5**
3	100	4	**48.7**	48.5
3	200	2	**106.7**	106.4
3	200	4	**97.9**	97.8
3	500	2	**154.9**	150.6
3	500	4	**237.6**	236.0
Avg.			**115.0**	112.8

m	n	$\|\Sigma\|$	LIMSBS-ensemble	IMSBS
5	50	2	**20.5**	**20.5**
5	50	4	**15.3**	**15.3**
5	100	2	32.6	**32.9**
5	100	4	28.0	**28.1**
5	200	2	**45.5**	42.1
5	200	4	**41.8**	38.8
5	500	2	**45.7**	39.2
5	500	4	**58.0**	45.6
10	50	2	**11.7**	**11.7**
10	50	4	7.3	**7.5**
10	100	2	**14.3**	12.0
10	100	4	**9.6**	8.9
10	200	2	**14.3**	12.8
10	200	4	**9.4**	8.2
10	500	2	**14.3**	13.2
10	500	4	**10.0**	8.6
Avg.			**23.6**	21.6

Table 2. Statistical comparison between LIMSBS-ensemble and IMSBS using the Wilcoxon signed-rank test (one-sided).

Comparison	Wins / Ties / Losses	p-value	Conclusion
All instances ($m = 2, 3, 5, 10$)	20 / 7 / 5	0.009	LIMSBS significantly better
Small instances ($m = 2, 3$)	10 / 4 / 2	< 0.01	Significant difference
Large instances ($m = 5, 10$)	10 / 3 / 3	< 0.05	LIMSBS significantly better

Table 3. Comparison of LIMSBS-ensemble and IMSBS on benchmark set REAL.

RAT instances

m	LIMSBS-ensemble	$t[s]$	IMSBS	$t[s]$	LCS*
10	**28**	43.44	25	30.65	209
15	**11**	23.39	**11**	24.26	189
20	**10**	11.31	9	49.36	174
25	**9**	17.53	7	29.62	173
40	**5**	18.18	4	48.64	154
60	**3**	22.62	1	45.22	154
80	**1**	22.52	**1**	53.02	144
100	**1**	25.71	**1**	44.30	139
150	**1**	39.51	**1**	66.57	131
200	**1**	39.82	**1**	62.54	126

VIRUS instances

m	LIMSBS-ensemble	$t[s]$	IMSBS	$t[s]$	LCS*
10	**87**	134.51	36	43.14	228
15	**19**	27.08	**19**	36.52	206
20	**11**	16.31	10	26.92	194
25	**13**	16.66	9	30.40	196
40	**7**	20.39	5	32.12	174
60	**5**	20.26	4	34.38	168
80	**4**	22.80	3	40.64	163
100	**3**	35.27	**3**	43.25	160
150	**4**	27.71	3	31.80	157
200	**1**	32.06	**1**	48.59	156

- Overall, LIMSBS-ensemble outperforms IMSBS in 15 out of 20 instances, while 5 instances result in ties and none favor IMSBS. This demonstrates a consistent dominance of LIMSBS-ensemble across the tested dataset.
- In terms of solution quality, LIMSBS-ensemble achieves an average value of 11.8, compared to 9.9 for IMSBS, corresponding to an average relative improvement of approximately 19%. The median quality values are 5 (LIMSBS-ensemble) and 4 (IMSBS), confirming that improvements are consistent and not driven by outliers.
- The advantage of LIMSBS-ensemble is particularly pronounced on more difficult instances (e.g., virus cases with higher variability), where substantial gains are observed (e.g., 87 vs. 36). In contrast, ties typically occur on simpler instances with very low objective values.

- Regarding computational time, IMSBS is generally faster, but achieving an average runtime of 36.9 seconds compared to 32.6 seconds for the ensemble LIMSBS-ensemble. However, this difference is not consistent across all instances, and in several cases LIMSBS-ensemble is competitive or even faster.
- When comparing the lengths of best (constrained-free) LCS from the literature, it is obvious that gap constraints, and specifically those included in this work, affecting final solutions significantly. For the instances with $m \geq 60$ in many cases found solutions are even trivial indicating the strong influence of a huge number of gap constraints in restricting search space of common subsequences fulfilling these.

The results demonstrate that the LIMSBS-ensemble provides a substantial and consistent improvement in solution quality over IMSBS on real-world instances. This makes it a preferable choice when solution quality is the main objective.

5 Conclusions

This study tackled a variant of the longest common subsequence problem which involves gap constraints for each input sequence, with applications in comparison of biological molecules and time-series analysis. We designed a learned heuristic, represented by the outputs of a neural network of predefined structure, whose weights are automatically set up by a biased random key genetic algorithm in the process of wights evolution. More advances heuristic is designed by combining the learned heuristic and the best performed hand-crafted heuristic from the literature, designing an ensemble heuristic. Further, this heuristic is incorporated into the best heuristic methodology, the iterative multi-source beam search, to guide the search. The experimental evaluation demonstrated that this approach outperformed all other competing approaches on the known synthetic problem instances from literature as well as newly introduced real problem instances that simulate energy-aware context between successive nucleotides via the data-driven gap constraints. Future work could involve the testing scenario for protein molecules, and instances with much larger sequence lengths to further asses the scalability od the designed approaches. Another task aims at learning the NN-based heuristic by some other meta-heuristic involving different fitness evaluators besides the utilized mean value while employing different stopping criteria in the training to prevent from over-fitting.

Acknowledgments. This publication is co- funded by the European Union's Horizon Europe research and innovation program under the Marie Sklodowska-Curie CO-FUND Postdoctoral Programme grant agreement No.101081355– SMASH and by the Republic of Slovenia and the European Union from the European Regional Development Fund. Co-funded by European Union. Views and opinions expressed are those of the authors only and do not necessarily reflect those of the European Union or European Research Executive Agency (REA). Neither the European Union nor the REA can be held responsible for them.

References

1. Adamson, D., Kosche, M., Koß, T., Manea, F., Siemer, S.: Longest common subsequence with gap constraints. In: International Conference on Combinatorics on Words. pp. 60–76. Springer (2023)
2. Amari, S.i.: Backpropagation and stochastic gradient descent method. Neurocomputing **5**(4-5), 185–196 (1993)
3. Bishop, C.M.: Neural networks and their applications. Review of scientific instruments **65**(6), 1803–1832 (1994)
4. Djukanovic, M., Balaban, N., Blum, C., Kartelj, A., Dzeroski, S., Zebec, Z.: On solving a generalized variable gapped longest common subsequence problem. In: Proceedings of the 20th International Conference on Computer Aided Systems Theory. Springer, 2026. Accepted
5. Djukanovic, M., Raidl, G., Blum, C.: Finding longest common subsequences: New anytime A* search results. Applied Soft Computing **95**, 106499 (2020)
6. Djukanović, M., Reixach, J., Nikolikj, A., Eftimov, T., Kartelj, A., Blum, C.: A learning search algorithm for the restricted longest common subsequence problem. Expert Systems with Applications **284**, 127731 (2025)
7. Gurney, K.: An introduction to neural networks. CRC press (2018)
8. Lafond, M., Lai, W., Liyanage, A., Zhu, B.: The longest subsequence-repeated subsequence problem. In: International Conference on Combinatorial Optimization and Applications. pp. 446–458. Springer (2023)
9. Lainscsek, C., Sejnowski, T.J.: Delay differential analysis of time series. Neural computation **27**(3), 594–614 (2015)
10. Lin, G., Chen, Z.Z., Jiang, T., Wen, J.: The longest common subsequence problem for sequences with nested arc annotations. Journal of Computer and System Sciences **65**(3), 465–480 (2002)
11. Mienye, I.D., Swart, T.G., Obaido, G.: Recurrent neural networks: A comprehensive review of architectures, variants, and applications. Information **15**(9), 517 (2024)
12. Peng, Y.H., Yang, C.B.: Finding the gapped longest common subsequence by incremental suffix maximum queries. Information and Computation **237**, 95–100 (2014)
13. Penga, Y.H., Yangb, C.B.: The longest common subsequence problem with variable gapped constraints. In: Proceedings of the 28th Workshop on Combinatorial Mathematics and Computation Theory, Penghu, Taiwan. pp. 17–23 (2011)
14. Pérez Cáceres, L., López-Ibáñez, M., Stützle, T.: An analysis of parameters of irace. In: European Conference on Evolutionary Computation in Combinatorial Optimization. pp. 37–48. Springer (2014)
15. Reixach, J., Blum, C., Djukanović, M., Raidl, G.R.: A neural network based guidance for a brkga: An application to the longest common square subsequence problem. In: European Conference on Evolutionary Computation in Combinatorial Optimization (Part of EvoStar). pp. 1–15. Springer (2024)
16. SantaLucia Jr, J.: A unified view of polymer, dumbbell, and oligonucleotide dna nearest-neighbor thermodynamics. Proceedings of the National Academy of Sciences **95**(4), 1460–1465 (1998)
17. Watkins Jr, N.E., SantaLucia Jr, J.: Nearest-neighbor thermodynamics of deoxyinosine pairs in dna duplexes. Nucleic acids research **33**(19), 6258–6267 (2005)
18. Wiseman, S., Rush, A.M.: Sequence-to-sequence learning as beam-search optimization. In: Proceedings of the 2016 conference on empirical methods in natural language processing. pp. 1296–1306 (2016)

19. Wu, Y., Song, W., Cao, Z., Zhang, J., Lim, A.: Learning improvement heuristics for solving routing problems. IEEE transactions on neural networks and learning systems **33**(9), 5057–5069 (2021)